COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Sep 16, 2026	**DATE SUBMITTED:**	Sep 17, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Seatwork 9.1 **Implementing Trees**

1. PROCEDURE OUTPUT

What procedures did you perform?

PROCEDURE:

Procedure that performed:

ILO A

Task Title & Task Objective

Screenshot of the code of BinaryTree Main

CPE21S4_BAL > Midterm > Seatworks > Seatwork 9.1 > main.cpp > main()

```cpp
#include <iostream>
#include "binaryTree.h"

int main()
{

    BinaryTree<char> *root = new BinaryTree<char>('A');
    root->insertChildLeft('B');
    root->insertChildRight('C');

    root->getLeftChild()->insertChildLeft('D');
    root->getLeftChild()->insertChildRight('E');
    root->getRightChild()->insertChildRight('D');

    std::cout << "The root: " << root->getRootVal() << std::endl;
    std::cout << "The left child of root A is: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The left child of node B is: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of node B is: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of root A is: " << root->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of node C is: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    root->getRightChild()->setRootVal('K');
    std::cout << "The NEW right child value of root A is: " << root->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

Screenshot of the code of BinaryTree Header	CPE21S4_BAL > Midterm > Seatworks > Seatwork 9.1 > C binaryTree.h > ...

```
1   #ifndef BINARYTREE_H
2   #define BINARYTREE_H
3
4   template <typename T>
5   class BinaryTree
6   {
7   private:
8       T key;
9
10      // pointers for left and right child
11      BinaryTree<T> *leftChild;
12      BinaryTree<T> *rightChild;
13
14  public:
15      // constructor
16      BinaryTree(T rootObj);
17
18      // insert a child on the left node
19      void insertChildLeft(T newNode);
20      void insertChildRight(T newNode);
21
22      BinaryTree<T> *getLeftChild();
23      BinaryTree<T> *getRightChild();
24
25      T getRootVal();
26      void setRootVal(T newNode);
27  };
28  // constructor
29  template <typename T>
30  BinaryTree<T>::BinaryTree(T rootObj)
31  {
32      key = rootObj;
33      leftChild = nullptr;
34      rightChild = nullptr;
35  }
```

```cpp
// insert child left
template <typename T>
void BinaryTree<T>::insertChildLeft(T newNode)
{
    if (leftChild == nullptr)
    {
        leftChild = new BinaryTree<T>(newNode);
    }
    else
    {
        BinaryTree<T> *newNode = new BinaryTree<T>(newNode);
        // move the curent left child below the new node
        newNode->leftChild = leftChild;
        leftChild = newNode;
    }
    leftChild = new BinaryTree<T>(newNode);
}
// insert child right
template <typename T>
void BinaryTree<T>::insertChildRight(T newNode)
{
    if (rightChild == nullptr)
    {
        rightChild = new BinaryTree<T>(newNode);
    }
    else
    {
        BinaryTree<T> *newNode = new BinaryTree<T>(newNode);
        // move the curent left child below the new node
        newNode->rightChild = rightChild;
        rightChild = newNode;
    }
    rightChild = new BinaryTree<T>(newNode);
}
template <typename T>
BinaryTree<T> *BinaryTree<T>::getLeftChild()
{
    return leftChild;
}
template <typename T>
BinaryTree<T> *BinaryTree<T>::getRightChild()
{
    return rightChild;
}
template <typename T>
T BinaryTree<T>::getRootVal()
{
```

```cpp
template <typename T>
T BinaryTree<T>::getRootVal()
{
    return key;
}
template <typename T>
void BinaryTree<T>::setRootVal(T newNode)
{
    key = newNode;
}
#endif
```

Output of the code	

```
The root: A
The left child of root A is: B
The left child of node B is: D
The right child of node B is: E
The right child of root A is: C
The right child of node C is: D
The NEW right child value of root A is: K
PS C:\C++ Projects>
```

Function	Screenshot	Observation/Analysis
Class that has an encapsulated variable and including a public variable		Class in header file that contains private and public variable such as (T key, BinaryTree<T> of left and right child) in private class and (BinaryTree, (as a constructor), insertChild left and right, get left and right child, get rootVal, and setRootVal) for public class functions that will be called out and used outside the class
insertChildLeft()		Insertchild function on the left tree was inserting a specific value of the getLeftChild function like string, int, float, char, double, etc. Below it was an if statement that checks if the left child does not have a value inside and if the

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree
{
private:
    T key;

    // pointers for left and right child
    BinaryTree<T> *leftChild;
    BinaryTree<T> *rightChild;

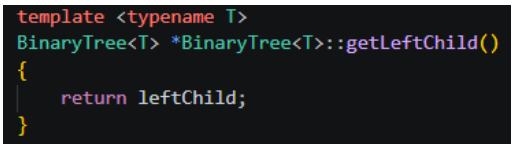
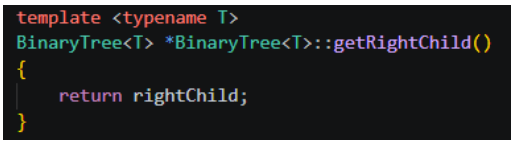
public:
    // constructor
    BinaryTree(T rootObj);

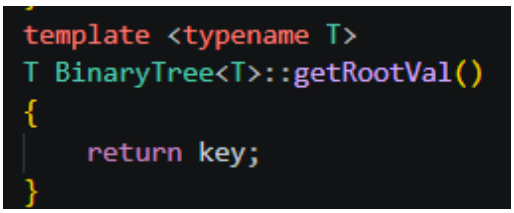
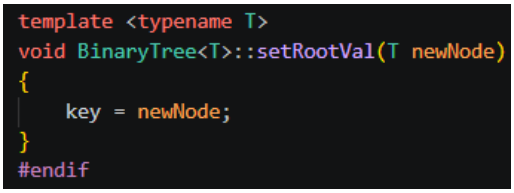
    // insert a child on the left node
    void insertChildLeft(T newNode);
    void insertChildRight(T newNode);

    BinaryTree<T> *getLeftChild();
    BinaryTree<T> *getRightChild();

    T getRootVal();
    void setRootVal(T newNode);
};
```

```cpp
// insert child left
template <typename T>
void BinaryTree<T>::insertChildLeft(T newNode)
{
    if (leftChild == nullptr)
    {
        leftChild = new BinaryTree<T>(newNode);
    }
    else
    {
        BinaryTree<T> *newnode = new BinaryTree<T>(newNode);
        // move the curent left child below the new node
        newnode->leftChild = leftChild;
        leftChild = newnode;
    }
    leftChild = new BinaryTree<T>(newNode);
}
```

		statement was correct, the leftChild function create a new key / value inside of it and if not, it moves the current leftchild tree below on the new key / value that have an empty space
insertChildRight()		Insertchild function on the right tree was inserting a specific value of the getRightChild function like string, int, float, char, double, etc. Below it was an if statement that checks if the right child does not have a value inside and if the statement was correct, the rightChild function create a new key / value inside of it and if not, it moves the current rightchild tree below on the new key / value that have an empty space
gotLeftChild()		Going to the left child tree and calling the getRootVal function to see the key inside the child tree
gotRightChild()		Going to the right child tree and calling the getRootVal function to see the key inside the child tree

getRootValue()		Getting the root value / key of a specific child tree but it depends on the user if it was left or right tree and also can check if the child has a value/key inside
setRootValue()		This function behavior of this function is changing the value/key inside of the root / main tree when it execute in the main driver code

ILO B

Task Title & Task Objective

Screenshot of the code	
Output of the code	
Analysis	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

SUPPLEMENTARY:

ILO C

Task Title & Objective:

Screenshot of the code	
Output of the code	
Analysis	

Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.